



Algorytmy i złożoność projekt №1 – “Eksperymentalne szacowanie złożoności obliczeniowej algorytmów sortowania”

Projekt został wykonany studentami grupy 4: Ratynskyi Volodymyr 71628 i Ivan Yehorov 71711

METRYKA DOKUMENTU

HISTORIA ZMIAN

Wersja: 1.0.5

Data wprowadzenia zmian: 14 kwietnia 2026 roku.

Wprowadzający zmianę: Ratynskyi Volodymyr, Ivan Yehorov

Opis zmiany: Poprawienie błędów gramatycznych.

OPRACOWANIE MERYTORYCZNE DOKUMENTU

Dokument opracowany przez: Prowadzący laboratorium.

Opracowujący: Volodymyr Ratynsky, Ivan Yehorov.

AKTUALIZACJE I PRZEGLĄDY

Częstotliwość aktualizacji dokumentu: Na żądanie prowadzącego laboratorium.

Odpowiedzialność za przegląd dokumentu: Prowadzący laboratorium / autorzy dokumentu.

ANOTACJA

Projekt dotyczy badania złożoności obliczeniowej wybranych algorytmów sortowania: BubbleSort, QuickSort, MergeSort oraz HeapSort. W ramach pracy zaimplementowano algorytmy, przeprowadzono testy dla różnych typów danych oraz rozmiarów tablic, a następnie dokonano analizy czasu wykonania, liczby porównań, zamian oraz zużycia pamięci. Wyniki zostały przedstawione w formie tabel i wykresów. Projekt został wykonany za pomocą języka programowania Java i bibliotek Java Swing oraz Java AWT.

Projekt został przetestowany na różnych komputerach, co spowodowało różnice w uzyskanych wynikach. Z tego powodu jest opisane charakterystyka sprzętowa komputera na którym zostały uzyskane wyniki:

- CPU: Intel Core i3-10100
- RAM: 8GB DDR4 jednokanalowa, z częstotliwością ~ 4000 MHz
- System Operacyjny: Windows 11 Pro
- Wersja Java: 21.0.2

SPIS TREŚCI

METRYKA DOKUMENTU	2
HISTORIA ZMIAN	2
OPRACOWANIE MERYTORYCZNE DOKUMENTU	2
AKTUALIZACJE I PRZEGLĄDY	2
ANOTACJA	3
SPIS TREŚCI	4
CEL DOKUMENTU	7
ADRESACI DOKUMENTU	7
OPIS UŻYWANYCH ALGORYTMÓW	8
BubbleSort	8
Opis działania	8
Złożoność czasowa i pamięciowa	8
Lista kroków	8
Implementacja algorytmu w projekcie	9
Wyniki testowania i wykresy	10
Wnioski	13
QuickSort	13
Opis działania	13
Złożoność czasowa i pamięciowa	14
Lista kroków	14
Implementacja algorytmu w projekcie	15
Wyniki testowania i wykresy	17
Wnioski	19

MergeSort	20
Opis działania	20
Złożoność czasowa i pamięciowa	21
Lista kroków	21
Implementacja algorytmu w projekcie	23
Wyniki testowania i wykresy	24
Wnioski	27
HeapSort.....	28
Opis działania	28
Złożoność czasowa i pamięciowa	28
Lista kroków	28
Implementacja algorytmu w projekcie	30
Wyniki testowania i wykresy	31
Wnioski	33
PORÓWNANIE ZASTOSOWANYCH ALGORYTMÓW	35
Porównanie czasu wykonania.....	35
Porównanie liczbę porównań	36
Porównanie liczbę zamian	37
Porównanie zużycia pamięci.....	38
Ocena wydajności algorytmów na podstawie wyników	38
ARCHITEKTURA PROGRAMU	40
Generowanie danych testowych	41
Zbieranie metryk	42
Testowanie algorytmów	43
Interfejs użytkownika.....	43

Generowanie wykresów	44
WNIOSKI PROJEKTU.....	46
ŹRÓDŁA.....	47

CEL DOKUMENTU

Celem dokumentu jest opisanie wykonania projektu, algorytmów oraz programu

ADRESACI DOKUMENTU

Dokument został przygotowany dla prowadzącego laboratorium

OPIS UŻYWANYCH ALGORYTMÓW

BubbleSort

Opis działania

BubbleSort jest jednym z najprostszych algorytmów sortowania, opartym na wielokrotnym porównywaniu sąsiednich elementów i zamianie ich miejscami, jeśli są w niewłaściwej kolejności.

Algorytm działa iteracyjnie, wykonując kolejne przejścia przez tablicę. Po każdym przejściu największy element z nieposortowanej części zostaje przesunięty na swoją docelową pozycję.

W najgorszym przypadku liczba porównań wynosi:

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

Złożoność czasowa i pamięciowa

W najlepszym przypadku, jeżeli tablica została posortowana, złożoność czasowa jest równa $O(n)$. W najgorszym złożoność będzie $O(n^2)$. Natomiast złożoność pamięciowa zawsze będzie $O(1)$.

Lista kroków

BubbleSort(A, n):

```
    for i ← 0 to n-2: // przechodzimy przez tablicę n-1 razy
        swapped ← false // zmienna sprawdzająca czy nastąpiła jakaś
        zamiana
        for j ← 0 to n-i-2: // przechodzimy przez nieposortowaną część tablicy
            if A[j] > A[j+1]: // porównanie sąsiednich elementów
                swap(A[j], A[j+1]) // zamiana
                swapped ← true // była zmiana
            fi
        endfor
    endfor
```

Implementacja algorytmu w projekcie

```
1 package Algorytmy;
2
3 import Metryki.Metryki;
4
5 public class BubbleSort implements AlgrorytmSortowania { 1 usage
6
7     public String getName() { 5 usages
8         return "BubbleSort";
9     }
10
11     public void sort(int[] arr, Metryki metryki) { 1 usage
12
13         for (int i = 0; i < arr.length - 1; i++) {
14
15             boolean swapped = false;
16
17             for (int j = 0; j < arr.length - i - 1; j++) {
18
19                 metryki.porownania++;
20
21                 if (arr[j] > arr[j + 1]) {
22                     int tmp = arr[j];
23                     arr[j] = arr[j + 1];
24                     arr[j + 1] = tmp;
25
26                     metryki.zamiany++;
27                     swapped = true;
28                 }
29
30                 metryki.pamiec += 1;
31             }
32
33             if (!swapped) {
34                 break;
35             }
36         }
37     }
38 }
```

Rysunek 1 - kod implementacji BubbleSort w projekcie

Pętla zewnętrzna(*for (int i = 0; i < arr.length - 1; i++)*) odpowiada za kolejne przejścia. Po każdym przebiegu największy element tablicy jest równy się elementu z końca tablicy.

Pętla wewnętrzna(*for (int j = 0; j < arr.length - i - 1; j++)*) porównuje elementy sąsiednie, zakres maleje wraz z *i*, ponieważ ostatnie *i* elementów są już posortowane.

Każda iteracja zwiększa licznik(*metryki.porownania++*), odpowiada liczbie operacji:

$$\frac{n(n-1)}{2}$$

Zamiana elementów się dzieje jeśli elementy są w złej kolejności(*if (arr[j] > arr[j + 1])*)

Liczenie zamian się dzieje kiedy każda zamiana odpowiada jednej operacji przesunięcia.

Liczenie pamięci się dzieje kiedy każda operacja traktowana jako jednostkowe zużycie pamięci(*metryki.pamiec++*). Nie jest to dokładny pomiar fizycznej pamięci, lecz model: $O(1)$.

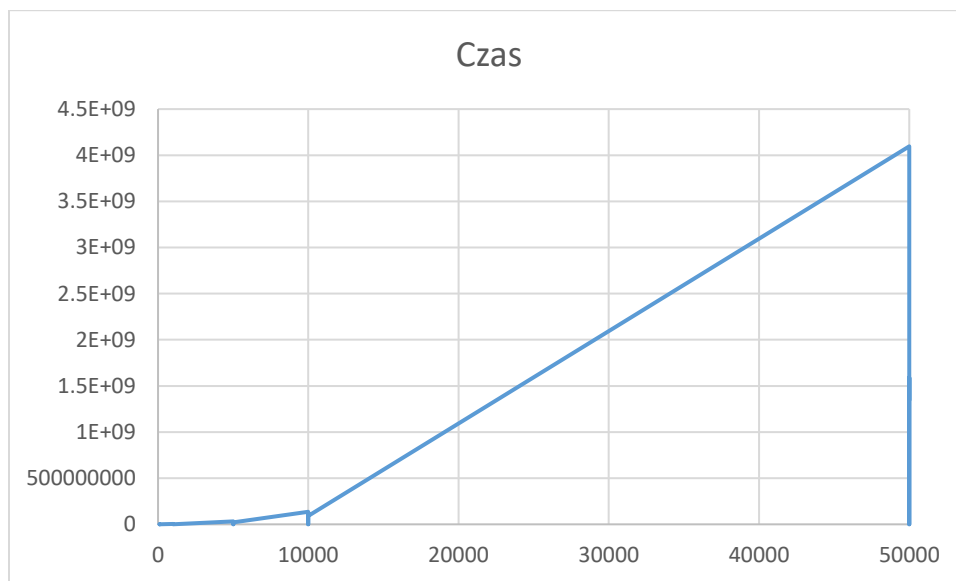
Ostatnie, jeśli w danym przejściu nie było zamian, to tablica jest posortowana. Poprawia najlepszy przypadek do $O(n)$.

Wyniki testowania i wykresy

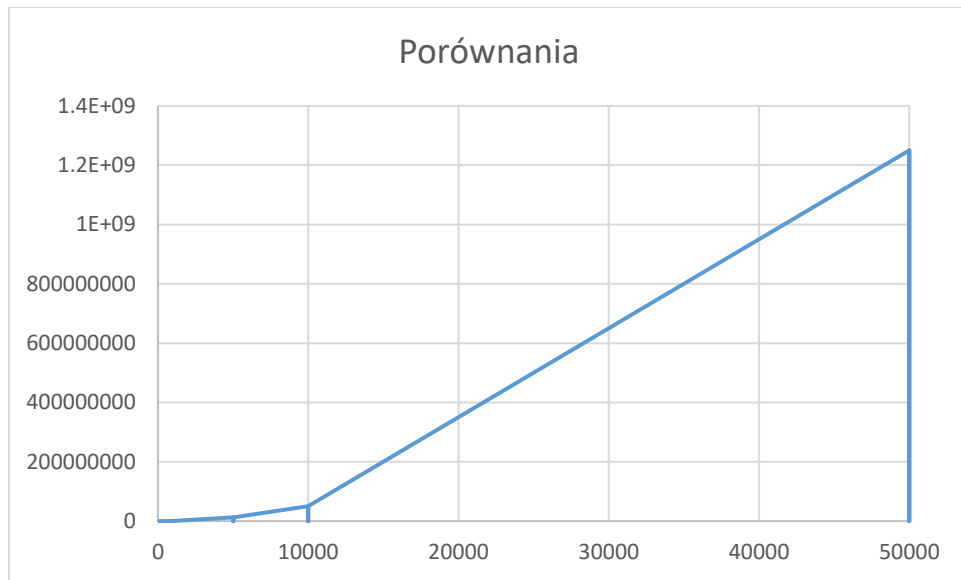
Poniżej przedstawiono wyniki pomiarów i grafiki dla różnych typów danych wejściowych oraz rozmiarów tablic

Algorytm	typ danych	n	czas	porównania	zamiany	pamięć
BubbleSort	losowe	100	185000	4814	2741	4814
BubbleSort	rosnace	100	3700	99	0	99
BubbleSort	malejace	100	201700	4950	4950	4950
BubbleSort	czesciowo	100	131100	4422	626	81768
BubbleSort	losowe	1000	3922900	499224	239834	499224
BubbleSort	rosnace	1000	31700	999	0	999
BubbleSort	malejace	1000	1965500	499500	499500	499500
BubbleSort	czesciowo	1000	1849600	498759	61096	498759
BubbleSort	losowe	5000	30507900	12497347	6234726	12497347
BubbleSort	rosnace	5000	18600	4999	0	4999
BubbleSort	malejace	5000	15630300	12497500	12497500	12497500
BubbleSort	czesciowo	5000	23706700	12472300	1451596	12472300
BubbleSort	losowe	10000	134884600	49994054	25043371	49994054
BubbleSort	rosnace	10000	119200	9999	0	9999
BubbleSort	malejace	10000	98645800	49995000	49995000	49995000
BubbleSort	czesciowo	10000	95677900	49950747	5589262	49950747
BubbleSort	losowe	50000	4104221200	1249948665	627131950	1249948665
BubbleSort	rosnace	50000	91900	49999	0	200016
BubbleSort	malejace	50000	1569326900	1249975000	1249975000	1249975000
BubbleSort	czesciowo	50000	1254257400	1249882765	144171071	1249882765

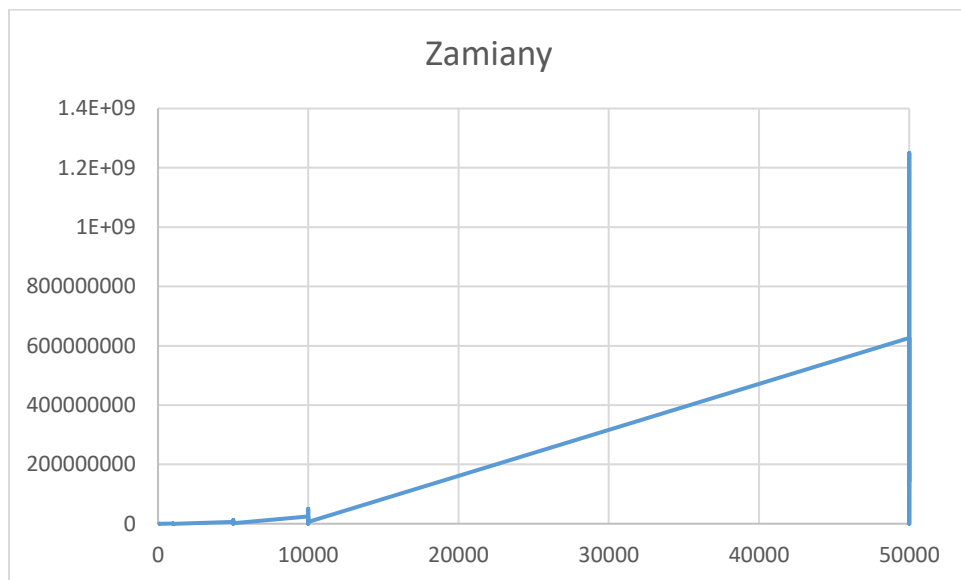
Rysunek 2 - Wyniki pomiarów dla algorytmu BubbleSort



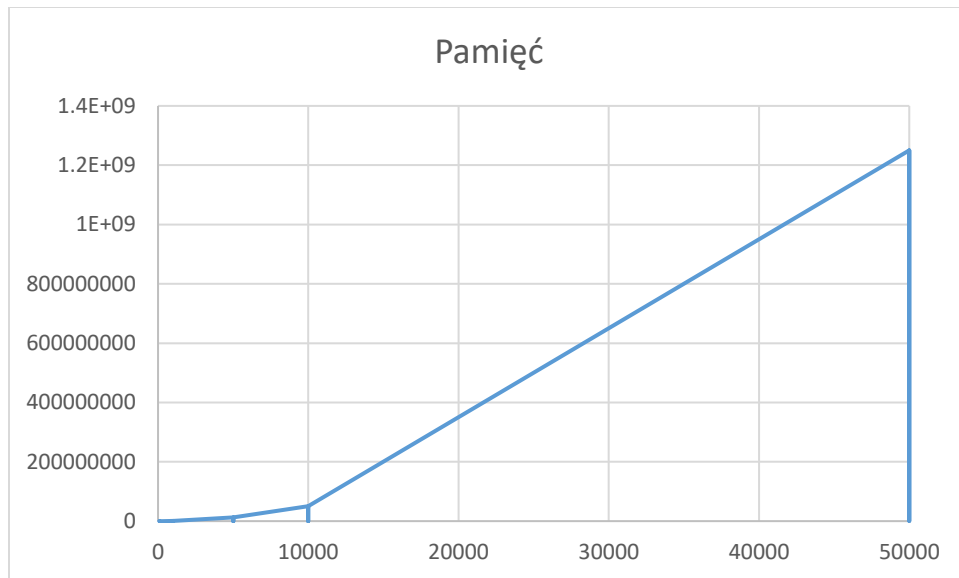
Rysunek 3 - Zależność czasu wykonania od rozmiaru danych dla BubbleSort



Rysunek 4 - Liczba porównań w zależności od rozmiaru danych



Rysunek 5 - Liczba zamian w zależności od rozmiaru danych



Rysunek 6 - Liczba zużytej pamięci w zależności od rozmiaru danych

Wnioski

Na podstawie testów można stwierdzić, że algorytm BubbleSort działa zgodnie z teorią i jego złożoność czasowa ma istotny wpływ na wyniki. Dla dużych rozmiarów danych jego wydajność znacząco spada, rośnie czas wykonania oraz zużycie pamięci.

W przypadku danych losowych i malejących liczba porównań oraz zamian rośnie bardzo szybko, co potwierdza jego niską efektywność. Algorytm nie nadaje się do dużych zbiorów danych.

Podsumowując, BubbleSort jest prosty w implementacji i dobrze ilustruje podstawowe zasady sortowania, jednak w praktyce ustępuje zaawansowanym algorytmom, używanymi w projekcie.

QuickSort

Opis działania

QuickSort jest algorytmem sortowania opartym na metodzie dziel i zwyciężaj. Jego działanie polega na wyborze elementu pivot, a następnie podziale tablicy na dwie części, elementy mniejsze od pivota oraz elementy większe od pivota. W projekcie pivot jest wybierany losowo, co zmniejsza ryzyko najgorszego przypadku.

Algorytm działa rekurencyjnie – po podziale tablicy, algorytm jest uruchamiany osobno dla lewej i prawej części, aż do momentu, gdy podtablice mają rozmiar 0 lub 1.

Kluczowym etapem jest procedura podzielenia, która wykonuje podział tablicy i ustawia pivot na właściwej pozycji.

W najgorzszym przypadku liczba operacji wynosi:

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

Co daje złożoność $O(n^2)$. To odbywa się jeżeli wybór pivotu został bardzo niekorzystny.

Złożoność czasowa i pamięciowa

Średnia oraz najlepsza złożoność czasowa wynosi $O(n \log n)$, ponieważ tablica jest dzielona na w miarę równe części, a tablica poziomów rekurencji wynosi $\log_2 n$.

W najgorzszym przypadku, gdy podział jest bardzo nierówny, złożoność czasowa wynosi $O(n^2)$.

Natomiast złożoność pamięciowa zależy od głębokości rekurencji i wynosi średnio $O(\log n)$, a w najgorzszym przypadku $O(n)$.

Lista kroków

QuickSort(A, low, high):

if low > high: // jeśli fragment tablicy ma więcej niż 1 element

*pivot ← Partition(A, low, high); // podział tablicy i wyznaczenie pozycji
 pivota*

*QuickSort(A, low, pivot – 1); // rekurencyjne sortowanie lewej części,
 mniejsze od pivota*

*QuickSort(A, pivot + 1, high); // rekurencyjne sortowanie prawej części,
 większe od pivota*

fi

Partition(A, low, high): // metoda dzielenia tablicy i wyznaczenie pozycji pivota

```

pivot ← A[high]; // wybór pivota
i ← low - 1; // indeks mniejszej części
for j ← low to high - 1: // przechodzenie przez elementy tablicy
    if A[j] < pivot: // porównanie elementu z pivotem
        i ← i + 1; // zwiększenie zakresu mniejszych elementów
        swap(A[i], A[j]); // zmiana elementów
    fi
endfor
swap(A[i+1], A[high]); // ustawienie pivota na właściwej pozycji
return i + 1; // zwrócenie indeksu pivota

```

Implementacja algorytmu w projekcie

```

10  public void sort(int[] arr, Metryki metryki) { 1 usage
11      quick(arr, low: 0, high: arr.length - 1, metryki);
12  }
13
14  private void quick(int[] arr, int low, int high, Metryki metryki) { 3 usages
15      if (low < high) {
16          int pi = partition(arr, low, high, metryki);
17          quick(arr, low, high: pi - 1, metryki);
18          quick(arr, low: pi + 1, high, metryki);
19          metryki.pamiec += 1; // symulacja stosu rekurencyjnie
20      }
21  }

```

Rysunek 8 - kod implementacji QuickSort w projekcie

```

23 ② private int partition(int[] arr, int low, int high, Metryki metryki) { 1 usage
24     Random rand = new Random();
25     int pivotIndex = low + rand.nextInt(bound: high - low + 1);
26
27     int tempPivot = arr[pivotIndex];
28     arr[pivotIndex] = arr[high];
29     arr[high] = tempPivot;
30
31     int pivot = arr[high];
32     int i = low - 1;
33
34     for (int j = low; j < high; j++) {
35         metryki.porownania++;
36         if (arr[j] < pivot) {
37             i++;
38             int tmp = arr[i];
39             arr[i] = arr[j];
40             arr[j] = tmp;
41             metryki.zamiany++;
42         }
43     }
44
45     int tmp = arr[i + 1];
46     arr[i + 1] = arr[high];
47     arr[high] = tmp;
48     metryki.zamiany++;
49
50     return i + 1;
51 }
52 }

```

Rysunek 7 - kod implementacji metody dzielenia tablicy

Metoda *quick* odpowiada za rekurencyjne sortowanie tablicy. Dzieli problem na mniejszy podproblemy.

if (low < high) sprawdza, czy fragment tablicy zawiera więcej niż jeden element. Jeśli tak, to wykonywane jest dalsze dzielenie.

Najważniejszym etapem jest wywołanie metody *partition*. Wybierany jest losowo pivot. Następnie pivot jest przenoszony na koniec tablicy, aby uprościć proces podziału.

Zmienna *i* oznacza granicę elementów mniejszych od pivota.

Pętla *for* przechodzi przez wszystkie elementy i porównuje je z pivotem. Jeśli *arr[j] < pivot*, to element trafia do lewej części. Wykonywana jest zamiana i zwiększany jest licznik *metryki.zamiany++* oraz dla porównań *metryki.porownania++*.

Po zakończeniu pętli pivot jest ustwierany na właściwej pozycji.

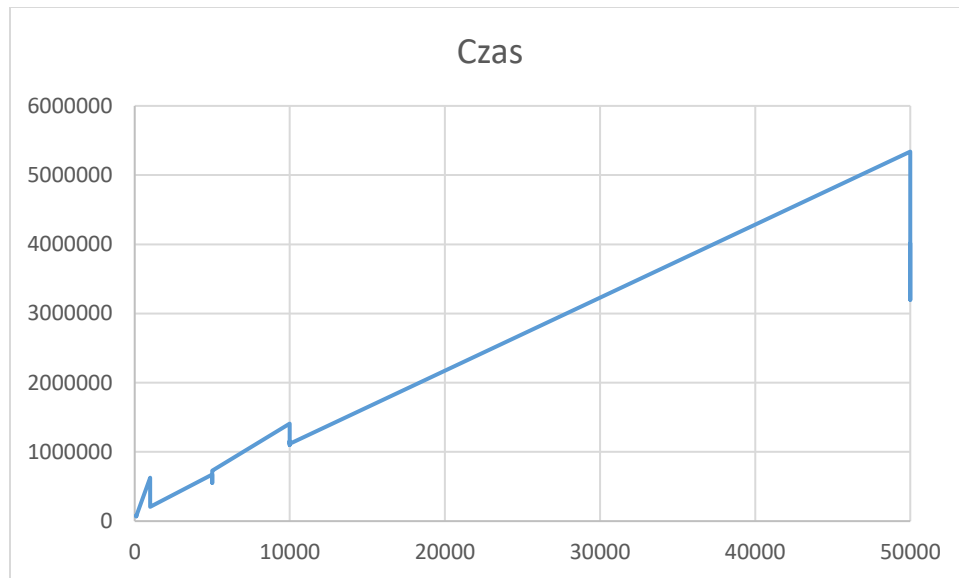
Po podziale tablicy algorytm wywołuje rekurencyjnie dla dwóch części. Każde wywołanie rekurencyjnie symuluje użycie stosu *metryki.pamiec++*.

Wyniki testowania i wykresy

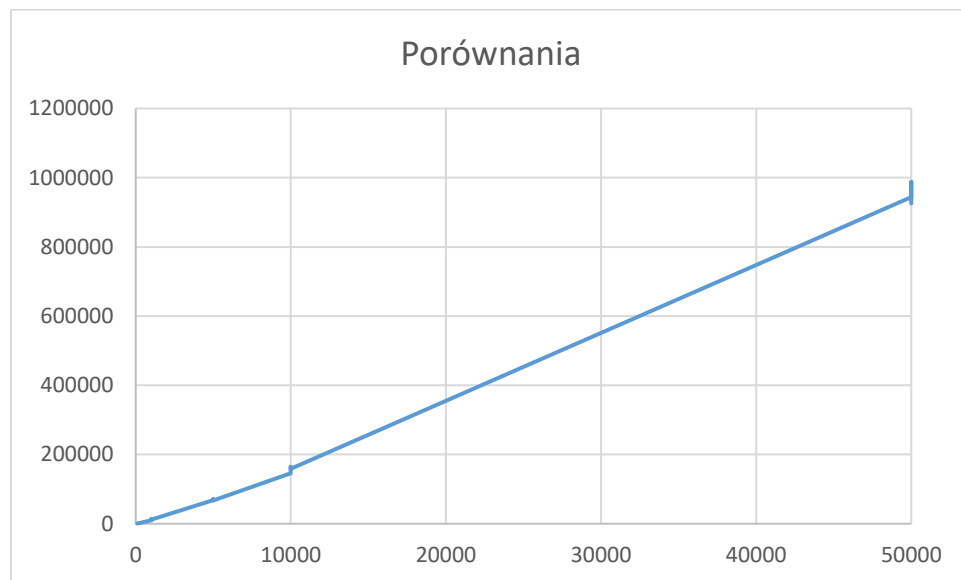
Poniżej przedstawiono wyniki pomiarów i grafiki dla różnych typów danych wejściowych oraz rozmiarów tablic.

Algorytm	typ danych	n	czas	porównania	zamiany	pamięć
QuickSort	losowe	100	88000	715	364	68
QuickSort	rosnace	100	76800	682	417	64
QuickSort	malejace	100	69200	700	464	66
QuickSort	czesciowo	100	70900	625	348	69
QuickSort	losowe	1000	626100	10204	6105	81264
QuickSort	rosnace	1000	323400	12480	8200	669
QuickSort	malejace	1000	247200	10571	5954	484424
QuickSort	czesciowo	1000	208900	11263	6300	673
QuickSort	losowe	5000	674900	67789	36983	3350
QuickSort	rosnace	5000	602900	71164	35976	482376
QuickSort	malejace	5000	551400	68815	37455	3357
QuickSort	czesciowo	5000	729700	67477	38323	81776
QuickSort	losowe	10000	1410500	146051	80687	522384
QuickSort	rosnace	10000	1101200	164664	89118	482376
QuickSort	malejace	10000	1152200	161426	97996	528240
QuickSort	czesciowo	10000	1119800	158400	85687	482368
QuickSort	losowe	50000	5339600	944008	483549	1130360
QuickSort	rosnace	50000	3196600	987906	515861	1048576
QuickSort	malejace	50000	3676000	957482	508657	1332976
QuickSort	czesciowo	50000	4017300	925584	506498	1048576

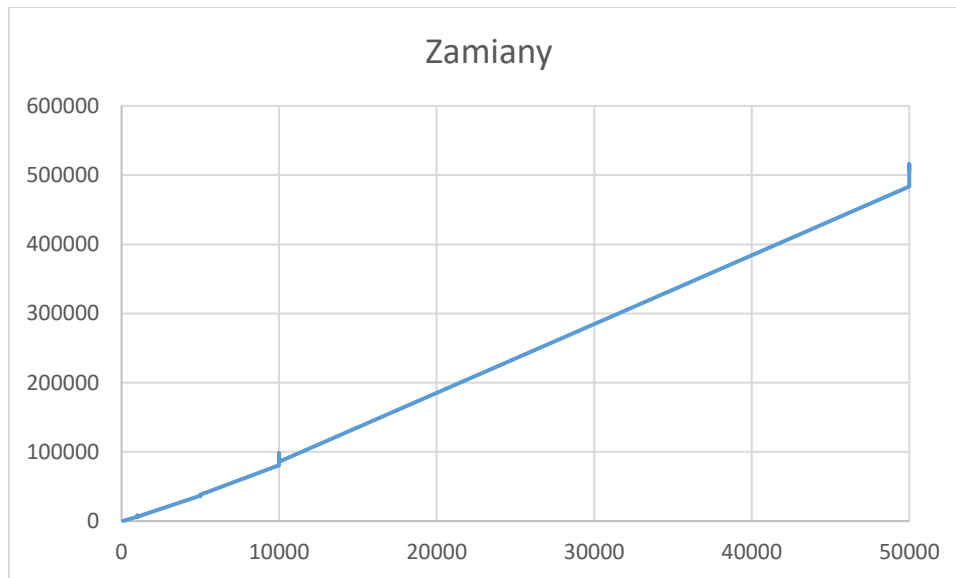
Rysunek 8 - Wyniki pomiarów dla algorytmu QuickSort



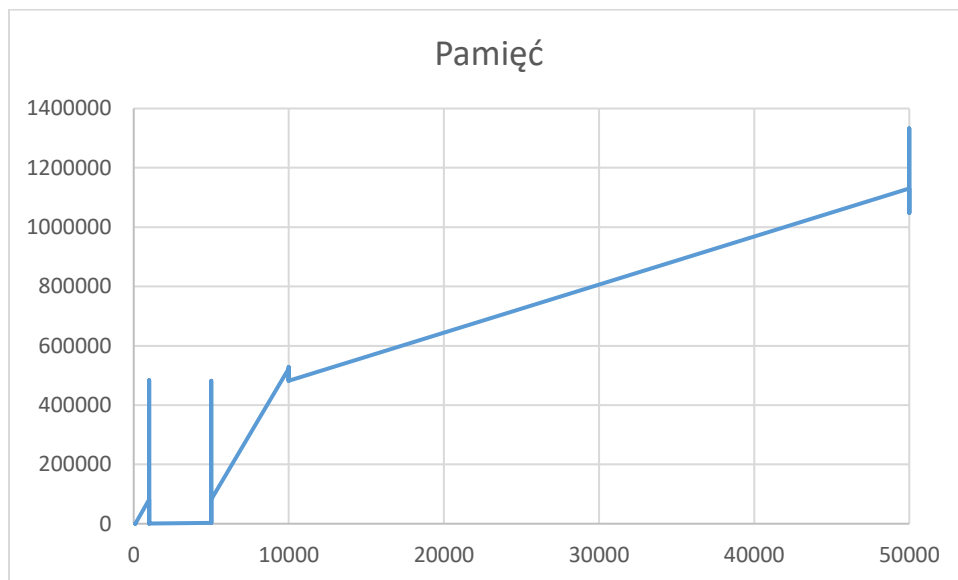
Rysunek 9 - Zależność czasu wykonania od rozmiaru danych dla QuickSort



Rysunek 10 - Liczba porównań w zależności od rozmiaru danych



Rysunek 11 - Liczba zamian w zależności od rozmiaru danych



Rysunek 12 - Liczba zużytej pamięci w zależności od rozmiaru danych

Wnioski

Na podstawie testów można stwierdzić, że algorytm QuickSort działa zgodnie z teorią. Czas wykonania rośnie zgodnie ze złożonością $O(n \log n)$, co potwierdza jego wysoką efektywność.

Zastosowanie losowego wyboru pivota zmniejsza ryzyko wystąpienia przypadku najgorszego $O(n^2)$.

Liczba porównań i zamian rośnie wolniej niż w algorytmach kwadratowych, na przykład BubbleSort. Jednocześnie algorytm wykorzystuje dodatkową pamięć wynikającą z rekurencji.

Podsumowując, QuickSort jest jednym z najbardziej efektywnych algorytmów zastosowanych w projekcie. Ma wysoką szybkość działania, dzięki czemu sprawdza się najlepiej przy dużych zbiorach danych.

MergeSort

Opis działania

MergeSort jest algorytmem sortowania opartym na metodzie, jak i QuickSort, dziel i zwyciężaj. Jego działanie jest podobne do QuickSorta, czyli dzieli tablicę na mniejsze części, aż do momentu, gdy każda podtablica zawiera jeden element. Następnie algorytm łączy te małe podtablice w większe, zachowując porządek elementów. Proces łączenia polega na porównywaniu elementów z dwóch posortowanych podtablic i wstawianiu mniejszego z nich do nowej tablicy wynikowej.

Jak i QuickSort, algorytm działa rekurencyjnie, najpierw dzieli problem na podproblemy, a następnie łączy ich wyniki w całość.

W przeciwieństwie do BubbleSort, MergeSort zawsze dzieli tablicę na połowy, dlatego liczba poziomów rekurencji wynosi:

$$\log_2 n$$

Na każdym poziomie wykonywane jest łączenie wszystkich elementów wynosi:

$$n$$

Całkowita liczba operacji wynosi złożoność:

$$O(n \log_2 n)$$

W najgorzym, średnim i najlepszym przypadku liczba operacji jest taka sama, ponieważ algorytm zawsze wykonuje pełny podział i łączenie niezależnie od danych wejściowych.

Złożoność czasowa i pamięciowa

Złożoność czasowa jest taka sama dla wszystkich przypadków danych wejściowych, ponieważ algorytm zawsze wykonuje pełny proces dzielenia i łączenia. Czyli we wszystkich przypadkach złożoność czasowa wynosi:

$$O(n \log_2 n)$$

Natomiast złożoność pamięciowa wynosi:

$$O(n)$$

ponieważ algorytm wymaga dodatkowej pamięci na tworzenie tablic dodatkowych podczas procesu łączenia.

Lista kroków

MergeSort(A):

if length(A) <= 1: // jeśli tablica ma 0 lub 1 element, to jest już posortowana

return A;

fi

mid ← length(A) / 2; // wyznaczenie środka tablicy

left ← A[0..mid-1]; // podział na lewą część

right ← A[mid..end]; // podział na prawą część

MergeSort(left); // rekurencyjne sortowanie lewej części

MergeSort(right); // rekurencyjne sortowanie prawej części

Merge(A, left, right); // łączenie dwóch posortowanych części do jednej tablicy

Merge(A, left, right):

$i \leftarrow 0$; // indeks dla lewej tablicy

$j \leftarrow 0$; // indeks dla prawej tablicy

$k \leftarrow 0$; // indeks dla tablicy wynikowej A

while $i < \text{length}(\text{left})$ and $j < \text{length}(\text{right})$ do: // dopóki są element w obu tablicach

if $\text{left}[i] \leq \text{right}[j]$: // porównanie elementów

$A[k] \leftarrow \text{left}[i]$; // przepisanie mniejszego elementu

$i \leftarrow i + 1$;

else:

$A[k] \leftarrow \text{right}[j]$;

$j \leftarrow j + 1$;

fi

$k \leftarrow k + 1$; // przejście do kolejnej pozycji

od

while $i < \text{length}(\text{left})$ do: // jeśli zostały elementy w lewej tablicy

$A[k] \leftarrow \text{left}[i]$;

$i \leftarrow i + 1$;

$k \leftarrow k + 1$;

od

while $j < \text{length}(\text{right})$ do: // jeśli zostały elementy w prawej tablicy

$A[k] \leftarrow \text{right}[j]$;

$j \leftarrow j + 1$;

$k \leftarrow k + 1$;

od

Implementacja algorytmu w projekcie

```
10  public void sort(int[] arr, Metryki metryki) { 1 usage
11     mergeSort(arr, metryki);
12 }
13
14 @ private void mergeSort(int[] arr, Metryki metryki) { 3 usages
15     if (arr.length < 2) return;
16
17     int mid = arr.length / 2;
18     int[] left = Arrays.copyOfRange(arr, from: 0, mid);
19     int[] right = Arrays.copyOfRange(arr, mid, arr.length);
20
21     metryki.pamiec += left.length + right.length;
22
23     mergeSort(left, metryki);
24     mergeSort(right, metryki);
25
26     merge(arr, left, right, metryki);
27 }
28
29 @ private void merge(int[] arr, int[] left, int[] right, Metryki metryki) { 1 usage
30     int i = 0, j = 0, k = 0;
31
32     while (i < left.length && j < right.length) {
33         metryki.porownania++;
34         if (left[i] <= right[j]) {
35             arr[k++] = left[i++];
36             metryki.zamiany++;
37         } else {
38             arr[k++] = right[j++];
39             metryki.zamiany++;
40         }
41     }
42
43     while (i < left.length) {
44         arr[k++] = left[i++];
45         metryki.zamiany++;
46     }
47
48     while (j < right.length) {
49         arr[k++] = right[j++];
50         metryki.zamiany++;
51     }
52 }
53 }
```

Rysunek 13 - kod implementacji MergeSort w projekcie

Metoda *mergeSort* odpowiada za podział tablicy oraz sterowanie całym procesem sortowania.

Na początku sprawdzany jest warunek *if(arr.length < 2)*, który kończy rekurencję, tablica jednoelementowa jest już posortowana.

Następnie tablica jest dzielona na dwie części. Tworzone są dwie nowe tablice dodatkowe, co powoduje zwiększenie zużycia pamięci. Odpowiada to złożoności pamięciowej $O(n)$.

Kolejnym krokiem jest rekurencyjne wywołanie sortowania dla lewej i prawej części. Powoduje to podział tablicy aż do poziomu pojedynczych elementów. Po zakończeniu podziału następuje łączenie dwóch posortowanych tablic.

Metoda *merge* odpowiada za sortowanie poprzez łączenie dwóch posortowanych tablic.

Zmienne *i, j, k* oznaczają indeksy odpowiednio dla tablic *left, right* oraz *arr*.

Pętla *while(i < left.length && j < right.length)* porównuje element z obu tablic. Jeśli element z lewej części jest mniejszy lub równy, to zostaje przepisany do tablicy wynikowej. W przeciwnym przypadku przepisany jest element z prawej części. Każde przypisanie jest traktowane jako operacja zamiany.

Po zakończeniu głównej pętli, pozostałe elementy jednej z tablic są kopiowane, co również zwiększa licznik zamiany.

Liczba operacji wynika z zależności:

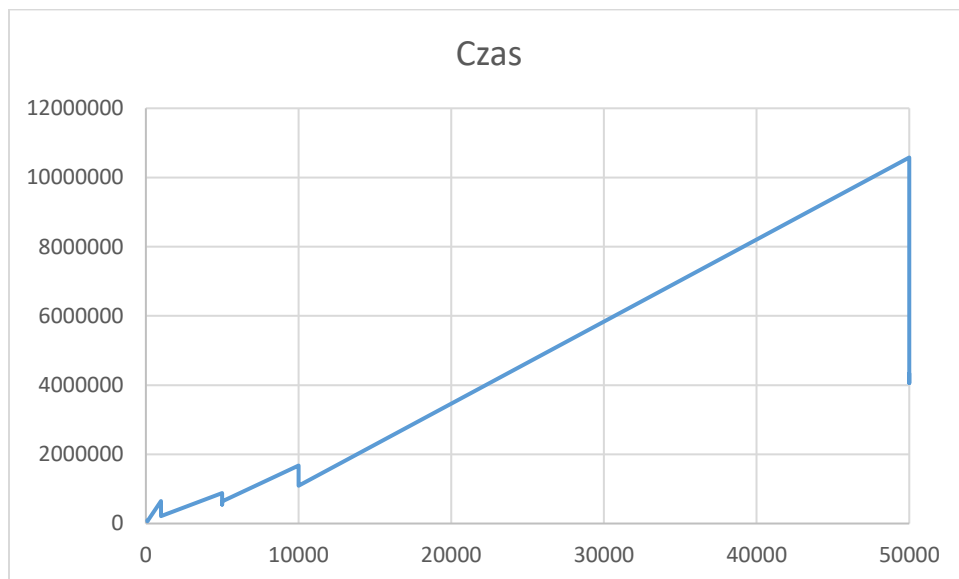
$$n \log n$$

Wyniki testowania i wykresy

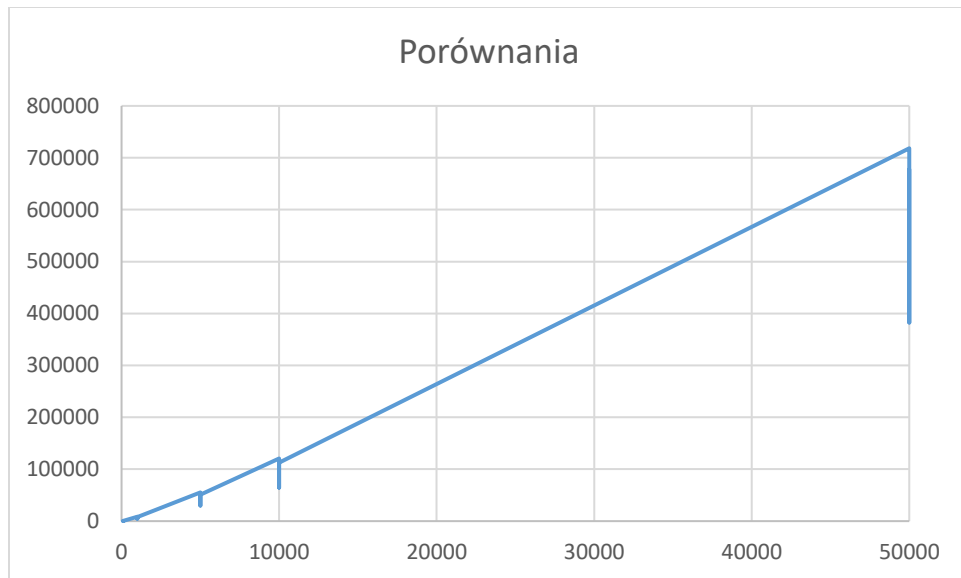
Poniżej przedstawiono wyniki pomiarów i grafiki dla różnych typów danych wejściowych oraz rozmiarów tablic.

Algorytm	typ danych	n	czas	porównania	zamiany	pamięć
MergeSort	losowe	100	91200	547	672	672
MergeSort	rosnace	100	77700	316	672	672
MergeSort	malejace	100	73100	356	672	672
MergeSort	czesciowo	100	65500	424	672	672
MergeSort	losowe	1000	651900	8704	9976	9976
MergeSort	rosnace	1000	455800	4932	9976	9976
MergeSort	malejace	1000	219200	5044	9976	482368
MergeSort	czesciowo	1000	216600	8056	9976	9976
MergeSort	losowe	5000	878100	55261	61808	566160
MergeSort	rosnace	5000	545200	29804	61808	482440
MergeSort	malejace	5000	560100	32004	61808	81712
MergeSort	czesciowo	5000	637200	51028	61808	502376
MergeSort	losowe	10000	1672200	120471	133616	1030688
MergeSort	rosnace	10000	1164400	64608	133616	1048520
MergeSort	malejace	10000	1093600	69008	133616	1048864
MergeSort	czesciowo	10000	1095200	112681	133616	1048272
MergeSort	losowe	50000	10577600	718303	784464	5242880
MergeSort	rosnace	50000	9206400	382512	784464	784464
MergeSort	malejace	50000	4060000	401952	784464	5385344
MergeSort	czesciowo	50000	4342000	677961	784464	5249080

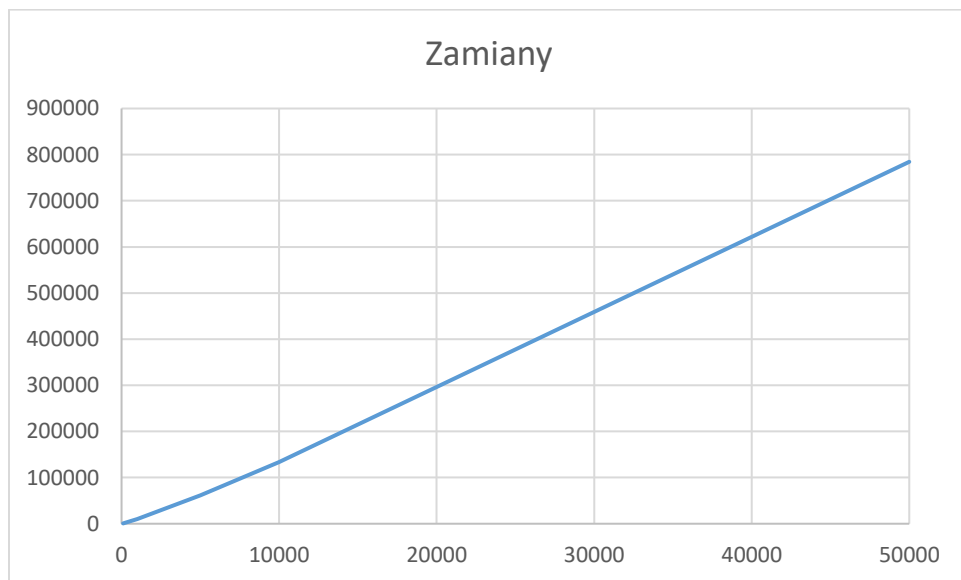
Rysunek 14 - Wyniki pomiarów dla algorytmu MergeSort



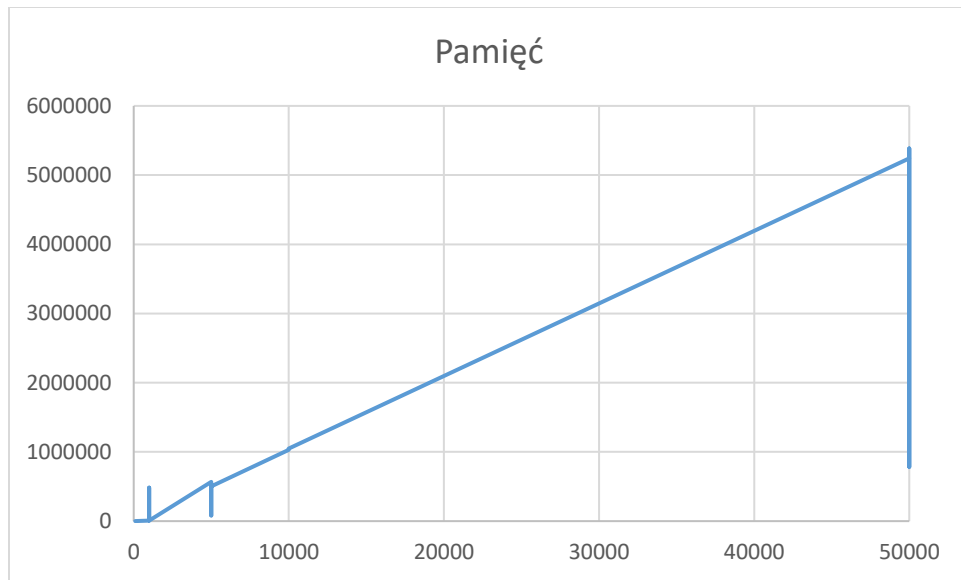
Rysunek 15 - Zależność czasu wykonania od rozmiaru danych dla MergeSort



Rysunek 16 - Liczba porównań w zależności od rozmiaru danych



Rysunek 17 - Liczba zamian w zależności od rozmiaru danych



Rysunek 18 - Liczba zużytej pamięci w zależności od rozmiaru danych

Wnioski

Na podstawie testów można stwierdzić, że algorytm MergeSort działa zgodnie z teorią i charakteryzuje się złożonością czasową $O(n \log_2 n)$, niezależnie od rodzaju danych wejściowych.

Liczba porównań i operacji pozostaje na podobnym poziomie dla różnych przypadków, co odróżnia MergeSort od algorytmów takich jak BubbleSort czy QuickSort w najgorzszym przypadku. Dzięki temu MergeSort jest efektywny przy dużych zbiorach danych.

Wadą algorytmu jest większe zużycie pamięci, wynikające z konieczności tworzenia dodatkowych tablic podczas procesu łączenia.

Podsumowując, MergeSort jest bardzo wydajnym algorytmem sortowania, który dobrze nadaje się dla dużych zbiorów danych, ale jest większe zużycie pamięci.

HeapSort

Opis działania

HeapSort jest algorytmem sortowania opartym na strukturze danych, która ma nazwę kopiec. Jego działanie polega na przekształceniu tablicy w kopiec, w którym każdy rodzic jest większy albo równy swoim dzieciom.

Najpierw budowany jest kopiec z całej tablicy. Następnie największy element jest zamieniany z ostatnim elementem tablicy i usuwany z kopca. Po każdej operacji, kopiec jest ponownie porządkowany. Proces powtarza aż do posortowania całej tablicy. Po każdym kroku największy element z nieposortowanej części trafia na swoją pozycję na końcu tablicy.

W najgorzym przypadku liczba operacji związanych z porównaniami i zamianami wynosi:

$$n \cdot \log_2 n$$

Złożoność czasowa i pamięciowa

W najlepszym przypadku złożoność czasowa algorytmu wynosi

$$O(n \log n)$$

ponieważ niezależnie od początkowego uporządkowania danych konieczne jest zbudowanie kopca oraz wykonanie operacji do jego naprawy.

W średnim oraz najgorzym przypadku złożoność czasowa zostaje taką samą, gdyż dla każdego elementu wykonywane są operacje o złożoności $\log n$.

Natomiast złożoność pamięciowa wynosi $O(1)$, ponieważ algorytm działa w miejscu i nie wymaga dodatkowej pamięci.

Lista kroków

HeapSort(A,n):

for $i \leftarrow (n/2) - 1$ to 0: // budowanie kopca maksymalnego z tablicy

Heapify(A,n,i); // naprawa kopca dla poddrzewa

```

endfor
for  $i \leftarrow n - 1$  to 1: // sortowanie właściwe
    swap( $A[0]$ ,  $A[i]$ ); // przeniesienie największego elementu na koniec
    Heapify( $A, i, 0$ ); // ponowne uporządkowanie kopca
endfor
Heapify( $A, n, i$ ):
    largest  $\leftarrow i$ ; // zakładamy, że największy jest korzeń
    left  $\leftarrow 2 * i + 1$ ; // lewe dziecko
    right  $\leftarrow 2 * i + 2$ ; // prawe dziecko
    if left < n and  $A[\text{left}] > A[\text{largest}]$ : // sprawdzenie czy lewe dziecko jest większe od
korzenia
        largest  $\leftarrow$  left;
    fi
    if right < n and  $A[\text{right}] > A[\text{largest}]$ : // sprawdzenie czy prawe dziecko jest
większe od korzenia
        largest  $\leftarrow$  right;
    fi
    if largest  $\neq i$ : // jeśli największy element nie jest korzeniem
        swap( $A[i]$ ,  $A[\text{largest}]$ ); // zamiana elementów

        Heapify( $A, n, \text{largest}$ ); // naprawa kopca za pomocy rekurencji
    fi

```

Implementacja algorytmu w projekcie

```
8  @ public void sort(int[] arr, Metryki metryki) { 1 usage
9      metryki.pamiec += arr.length;
10     int n = arr.length;
11
12     for (int i = n / 2 - 1; i >= 0; i--)
13         heapify(arr, n, i, metryki);
14
15     for (int i = n - 1; i > 0; i--) {
16         int temp = arr[0];
17         arr[0] = arr[i];
18         arr[i] = temp;
19         metryki.zamiany++;
20
21         heapify(arr, i, 0, metryki);
22     }
23 }
24
25 private void heapify(int[] arr, int n, int i, Metryki metryki) { 3 usages
26     int largest = i;
27     int l = 2 * i + 1;
28     int r = 2 * i + 2;
29
30     if (l < n) {
31         metryki.porownania++;
32         if (arr[l] > arr[largest]) largest = l;
33     }
34
35     if (r < n) {
36         metryki.porownania++;
37         if (arr[r] > arr[largest]) largest = r;
38     }
39
40     if (largest != i) {
41         int swap = arr[i];
42         arr[i] = arr[largest];
43         arr[largest] = swap;
44         metryki.zamiany++;
45
46         heapify(arr, n, largest, metryki);
47     }
48 }
49 }
```

Rysunek 19 - Kod implementacji HeapSort w projekcie

Metoda *sort* odpowiada za całe sortowanie tablicy przy użyciu kopca największego. Na początku jest zwiększany licznik pamięci dla symulowania wykorzystanej pamięci.

Pierwszym etapem jest budowanie kopca. Pętla *for(int i = n/2 - 1; i >= 0; i--)* przechodzi przez elementy od środka tablicy do początku i wywołuje metodę *heapify*.

Drugim etapem jest właściwe sortowanie. Pętla *for(int i = n - 1; i > 0; i--)* zamienia pierwszy element z ostatnim elementem nie posortowanej części tablicy. Po zamianie największy element trafia na końcową pozycję.

Po każdej zamianie jest wywoływana metoda *heapify*, która przywraca własności kopca dla pozostałej części tablicy.

Metoda *heapify* za strukturę kopca. Dla każdego dziecka jest wykonywane porównanie. Jeśli któreś z dzieci jest większe od aktualnego elementu, aktualizowana jest zmienna *largest*. Jeśli największy element nie jest korzeniem, wykonywana jest zamiana, a następnie metoda *heapify* jest wywoływana rekurencyjnie dla zmienionego drzewa.

Liczba operacji wynika z zależności:

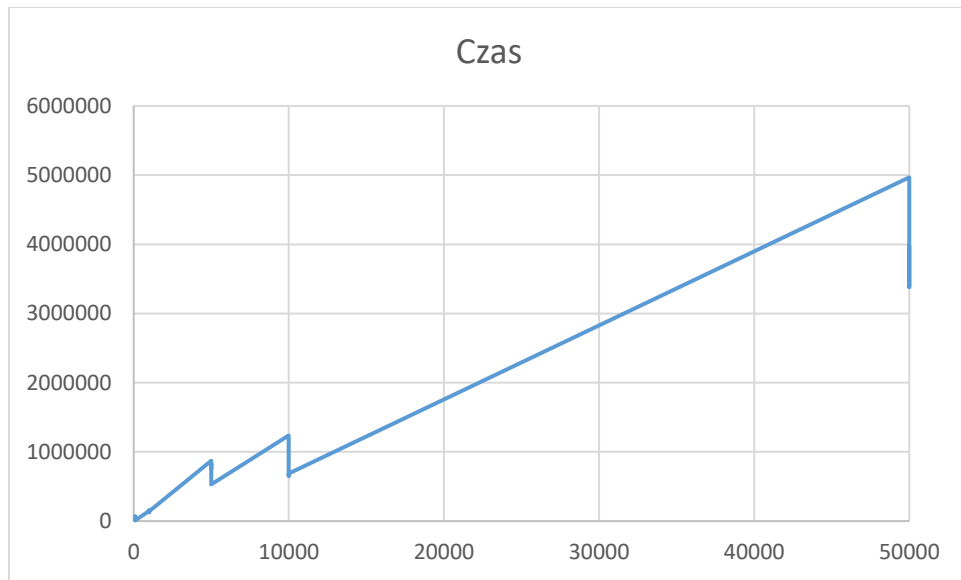
$$n \log n$$

Wyniki testowania i wykresy

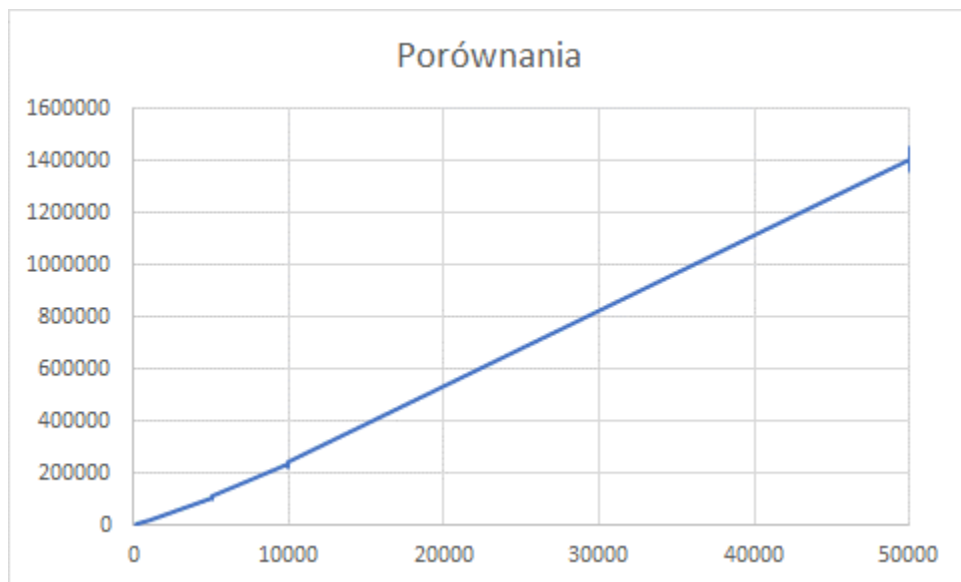
Poniżej przedstawiono wyniki pomiarów i grafiki dla różnych typów danych wejściowych oraz rozmiarów tablic.

Algorytm	typ danych	n	czas	porównania	zamiany	pamięć
HeapSort	losowe	100	71600	1023	575	100
HeapSort	rosnace	100	20300	1081	640	100
HeapSort	malejace	100	11900	944	516	100
HeapSort	czesciowo	100	12400	1065	620	100
HeapSort	losowe	1000	148500	16840	9067	1000
HeapSort	rosnace	1000	157700	17583	9708	1000
HeapSort	malejace	1000	128900	15965	8316	1000
HeapSort	czesciowo	1000	142700	17418	9584	1000
HeapSort	losowe	5000	871200	107614	57128	5000
HeapSort	rosnace	5000	757400	112126	60932	5000
HeapSort	malejace	5000	837200	103227	53436	5000
HeapSort	czesciowo	5000	530000	111208	60012	5000
HeapSort	losowe	10000	1236400	235600	124355	10000
HeapSort	rosnace	10000	953000	244460	131956	10000
HeapSort	malejace	10000	653700	226682	116696	10000
HeapSort	czesciowo	10000	689700	242518	130102	10000
HeapSort	losowe	50000	4967500	1410205	737664	200016
HeapSort	rosnace	50000	3506100	1455438	773304	200016
HeapSort	malejace	50000	3383300	1366047	698892	200016
HeapSort	czesciowo	50000	3978300	1446761	767516	248512

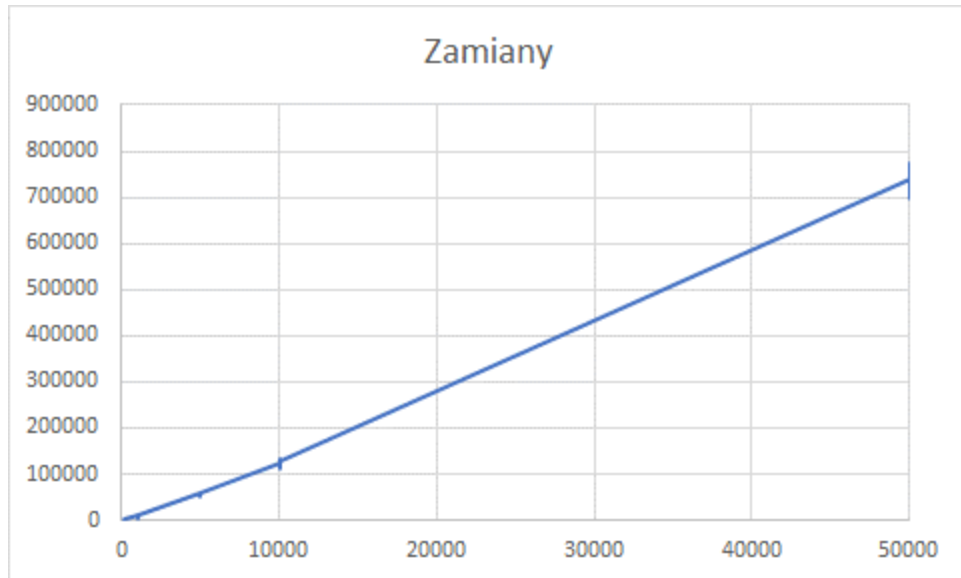
Rysunek 20 - Wyniki pomiarów dla algorytmu HeapSort



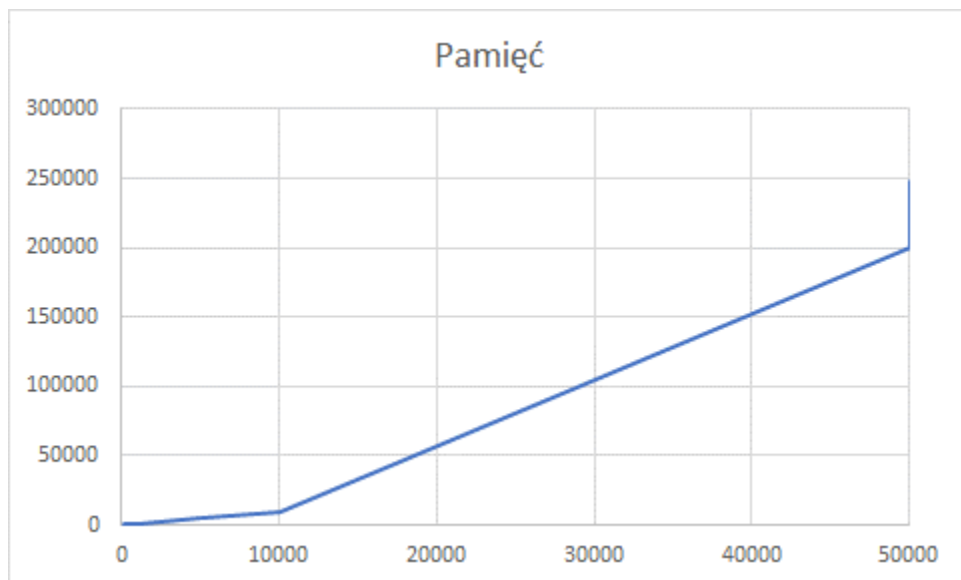
Rysunek 21 - Zależność czasu wykonania od rozmiaru danych dla HeapSort



Rysunek 22 - Liczba porównań w zależności od rozmiaru danych



Rysunek 23 - Liczba zamian w zależności od rozmiaru danych



Rysunek 24 - Liczba zużytej pamięci w zależności od rozmiaru danych

Wnioski

Na podstawie przeprowadzonych testów można stwierdzić, że algorytm działa zgodnie z teorią i charakteryzuje się złożonością czasową $O(n \log n)$ dla wszystkich przypadków danych wejściowych. Czas wykonania rośnie proporcjonalnie do $n \log n$, co potwierdza jego wydajność niezależnie od danych wejściowych.

Liczba porównań i zamian utrzymuje się na podobnym poziomie dla różnych typów danych, co odróżnia HeapSort od algorytmów takich jak BubbleSort. W porównaniu do QuickSort, HeapSort nie posiada przypadku najgorszego o złożoności $O(n^2)$, dzięki czemu jest bardzo oczekiwany.

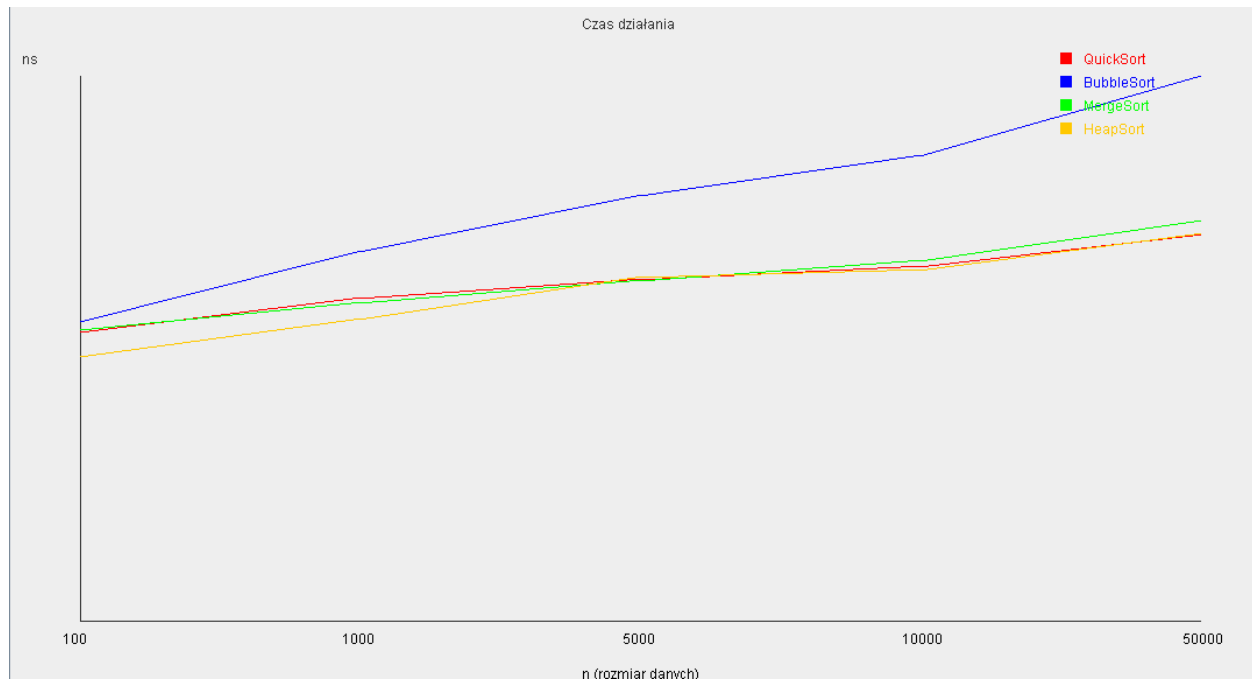
Dodatkową zaletą algorytmu jest niskie zużycie pamięci $O(1)$, ponieważ działa on w miejscu i nie wymaga dodatkowych struktur danych, w porównaniu do MergeSort.

Podsumowując, algorytm jest efektywnym do sortowania, który dobrze nadaje się dla dużych zbiorów danych, osobiwo w sytuacjach, gdzie ważne jest ograniczenie zużycia pamięci.

PORÓWNANIE ZASTOSOWANYCH ALGORYTMÓW

W tej części wykonano porównanie zastosowanych algorytmów sortowania, czyli BubbleSort, QuickSort, MergeSort oraz HeapSort. Analiza opiera się na wynikach, obejmujących czas wykonania, liczbę porównań, zamian oraz pamięci.

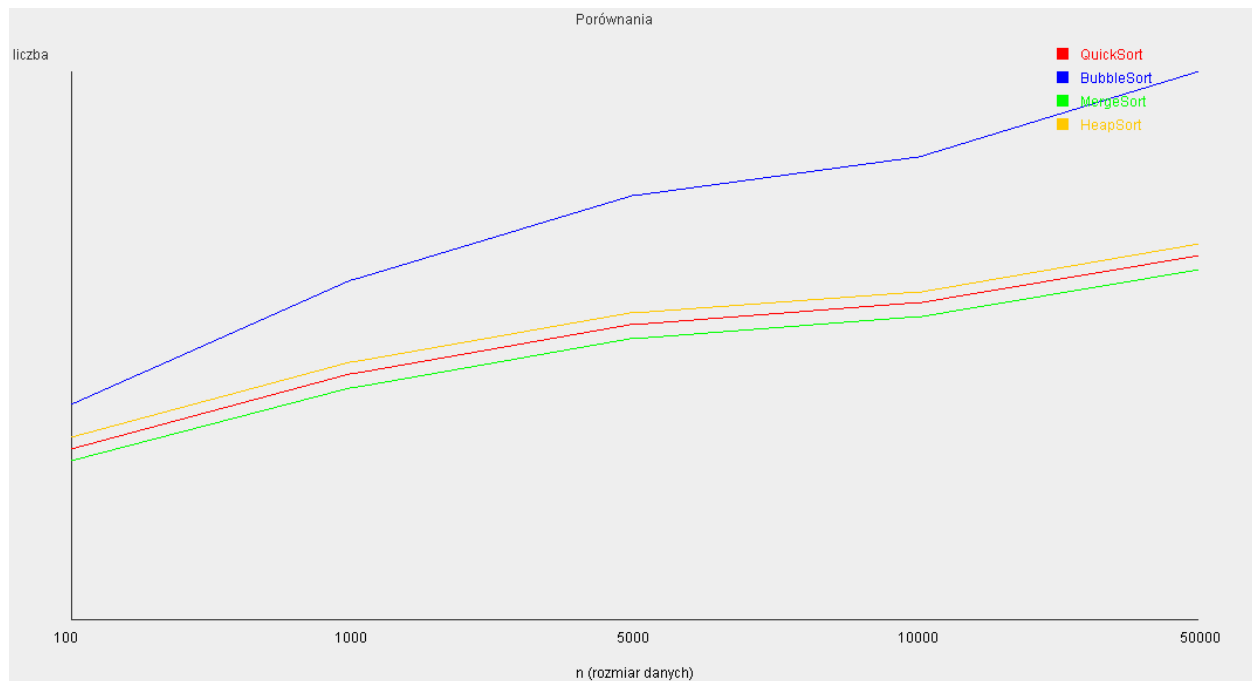
Porównanie czasu wykonania



Rysunek 25 - Wykres porównający algorytmy pod względem czasu w zależności od rozmiaru danych

Na podstawie wykresu można zauważyć że BubbleSort ma najgorszy czas wykonania, rośnie bardzo szybko wraz ze wzrostem danych. QuickSort, MergeSort oraz HeapSort mają lepszą wydajność, czas rośnie zgodnie z $O(n \log n)$. QuickSort jest najszybszy. HeapSort jest trochę wolniejszy od QuickSort, ale bardziej stabilny. MergeSort też ma stabilny czas, ale jest wolniejszy przez operacje kopiowanie tablic.

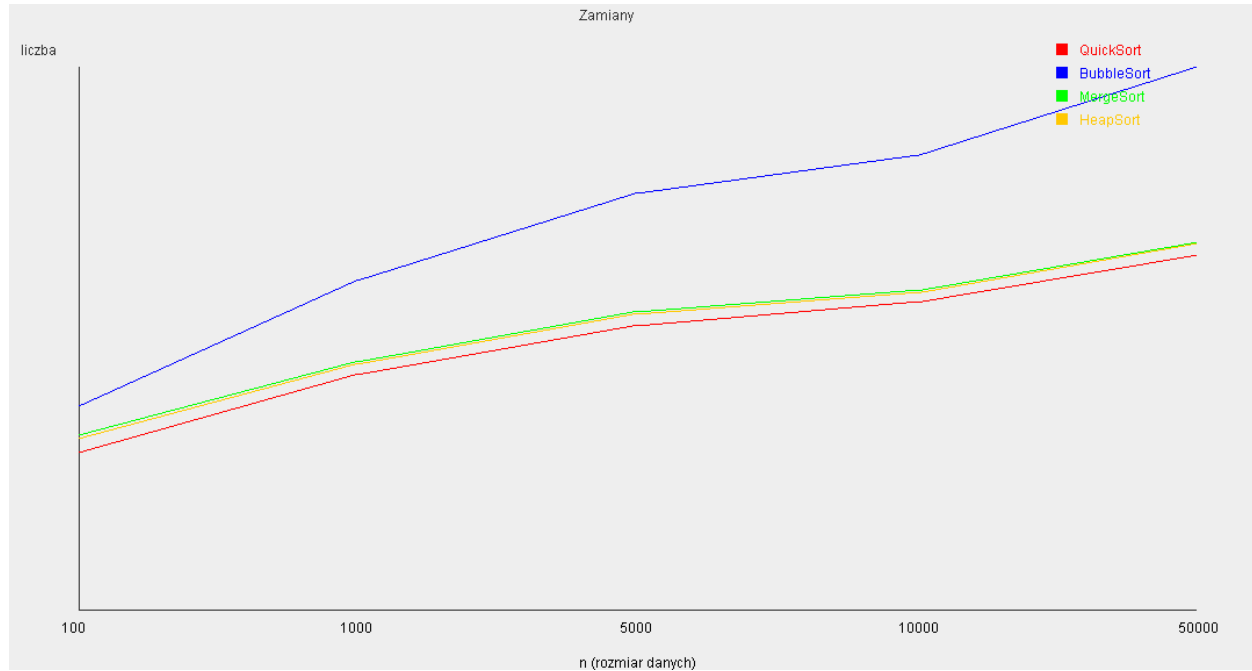
Porównanie liczbę porównań



Rysunek 26 - Wykres porównający algorytmy pod względem liczbę porównań w zależności od rozmiaru danych

Na podstawie wykresu można zobaczyć że BubbleSort wykonuje największą liczbę porównań. QuickSort ma mniejszą liczbę porównań niż BubbleSort, ale może wzrosnąć w złym przypadku. MergeSort i HeapSort mają dobrą liczbę porównań zgodnie z $n \log n$. HeapSort często wykonuje więcej porównań niż MergeSort, przez strukturę kopca.

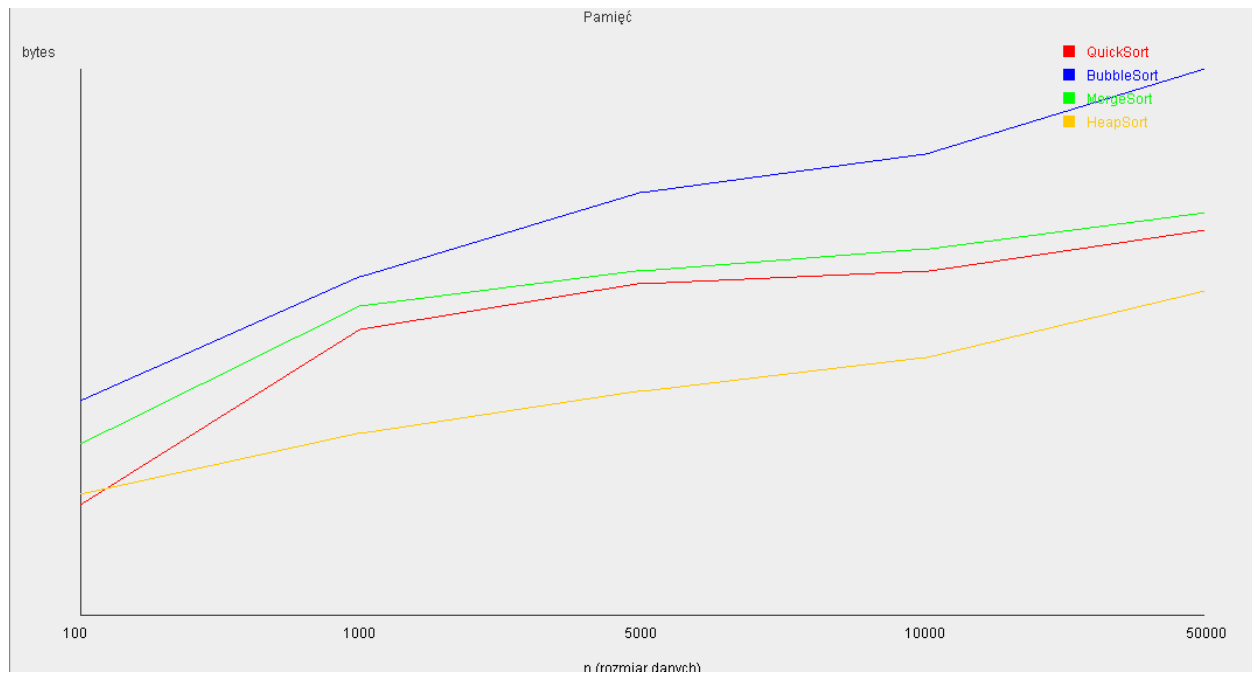
Porównanie liczbę zamian



Rysunek 27 - Wykres porównający algorytmy pod względem liczbę zamian w zależności od rozmiaru danych

Na podstawie wykresu można zauważyć że BubbleSort wykonuje dużo zamian, czyli ma najgorzy wynik. QuickSort ma umiarkowaną liczbę zamian. MergeSort operuje bardziej na kopiowaniu niż na zamianach. HeapSort wykonuje zamiany po każdym wykonaniu metody heapify, ale mniej niż BubbleSort i MergeSort.

Porównanie zużycia pamięci



Rysunek 28 - Wykres porównujący algorytmy pod względem liczbę zużytej pamięci w zależności od rozmiaru danych

Na podstawie wykresu, BubbleSort i HeapSort działają w miejscu i mają złożoność pamięciową $O(1)$, ale Bubble ma największą liczbę zużytej pamięci, a HeapSort potrzebuje najmniejszą liczbę pamięci. QuickSort ma średnią liczbę zużytej pamięci dlatego że działa rekurencyjnie i ma złożoność $O(\log n)$. MergeSort ma prawie największe zużycie pamięci.

Ocena wydajności algorytmów na podstawie wyników

Najlepszą wydajność posiada QuickSort, osiąga najkrótsze czasy wykonania dla różnych typów danych. Liczba operacji rośnie zgodnie z $O(n \log n)$, co robi go najlepszym wyborem dla dużych zbiorów danych.

HeapSort ma trochę niższą wydajność niż QuickSort. Działa stabilnie w czasie $O(n \log n)$, ale przez częste wykonanie metody heapify jest wolniejszy. Jego zaletą jest małe zużycie pamięci.

MergeSort ma dobrą wydajność czasową, ale gorszą pamięciową. Czas działania jest stabilny, jednak tworzenie dodatkowych tablic powoduje większe zużycie pamięci, co obniża go wydajność w porównaniu do QuickSort.

Najgorszą wydajność ma BubbleSort. Czas działania $O(n^2)$ powoduje bardzo szybki wzrost czasu dla dużych danych. W przeprowadzonych pomiarach widać, że wykonuje największą liczbę operacji oraz ma największe zużycie pamięci.

ARCHITEKTURA PROGRAMU

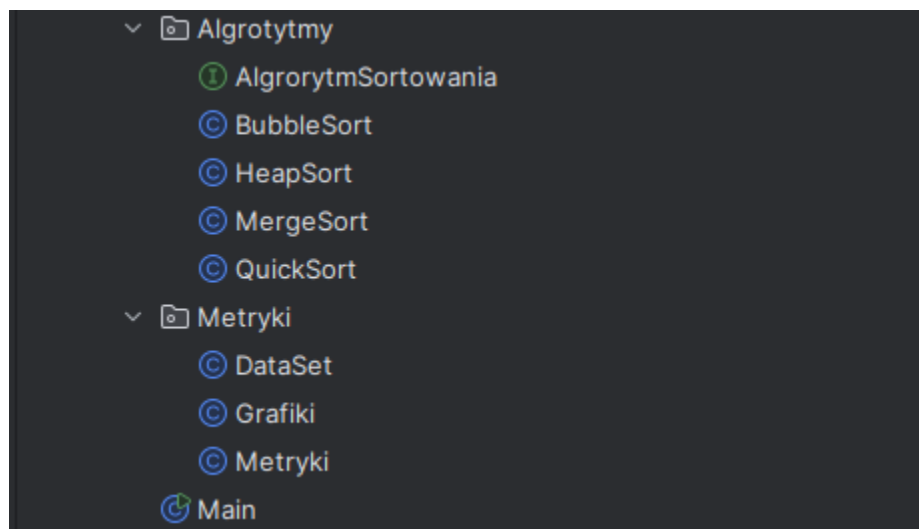
Program został napisany w języku Java. Składa się z kilku głównych pakietów.

Pakiet *Algorytmy* zawiera implementacje różnych algorytmów sortowania.

Pakiet *Metryki* odpowiada za zbieranie danych pomiarowych oraz generowanie danych.

Główna klasa *Main* steruje działaniem programu oraz interfejsem użytkownika.

Dodatkowo program wykorzystuje interfejs *AlgorytmSortowania*, który zawiera wywołanie wszystkich algorytmów.



Rysunek 28 - Struktura programu

Każdy algorytm implementuje interfejs *AlgorytmSortowania*. Dzięki temu wszystkie algorytmy mają jednolitą strukturę, można je łatwo porównywać w jednej pętli oraz możliwe jest pobieranie metryk w taki sam sposób.

```

1 package Algorytmy;
2
3 import Metryki.Metryki;
4
5 public interface AlgrorytmSortowania { 6 usages 4 implementations
6     String getName(); 6 usages 4 implementations
7     void sort(int[] array, Metryki metryki); 1 usage 4 implementations
8 }
9

```

Rysunek 29 - Kod interfejsu AlgorytmSortowania

Generowanie danych testowych

Klasa *DataSet* generuje różne typy danych, dane losowe, dane rosnące, dane malejące oraz dane częściowo posortowane. Pozwala to przetestować algorytmy w różnych warunkach.

```

1 package Metryki;
2
3 import java.util.Random;
4
5 public class DataSet { 4 usages
6     @ public static int[] random(int n) { 1 usage
7         Random r = new Random();
8         int[] arr = new int[n];
9         for (int i = 0; i < n; i++) arr[i] = r.nextInt( bound: 100000);
10        return arr;
11    }
12
13    @ public static int[] ascending(int n) { 2 usages
14        int[] arr = new int[n];
15        for (int i = 0; i < n; i++) arr[i] = i;
16        return arr;
17    }
18
19    @ public static int[] descending(int n) { 1 usage
20        int[] arr = new int[n];
21        for (int i = 0; i < n; i++) arr[i] = n - i;
22        return arr;
23    }
24
25    @ public static int[] partiallySorted(int n) { 1 usage
26        int[] arr = ascending(n);
27        Random r = new Random();
28        for (int i = 0; i < n / 10; i++) {
29            int a = r.nextInt(n);
30            int b = r.nextInt(n);
31            int tmp = arr[a];
32            arr[a] = arr[b];
33            arr[b] = tmp;
34        }
35        return arr;
36    }
37 }
38

```

Rysunek 30 - Implementacja klasy DataSet

Zbieranie metryk

Klasa *Metryki* przechowuje liczbę porównań i zamian oraz liczbę zużytej pamięci. Każdy algorytm wykorzystuje te wartości podczas działania, co pozwala na analizę jego wydajność.

```

1 package Metryki;
2
3 public class Metryki { 17 usages
4     public long porownania = 0; 8 usages
5     public long zamiany = 0; 12 usages
6     public long pamiec = 0; 5 usages
7 }
8

```

Rysunek 31 - Implementacja klasy Metryki

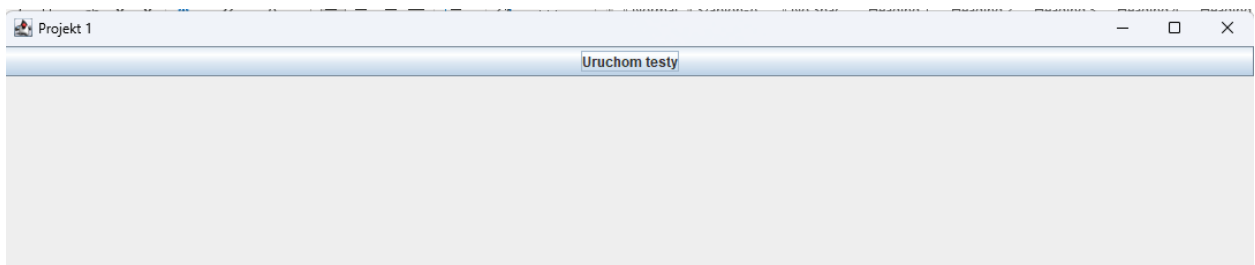
Testowanie algorytmów

Główna logika znajduje się w metodzie *runTests*. Dla każdego algorytmu, dla każdego rozmiaru danych, dla każdego typu danych generowana jest tablica, wykonywane jest sortowanie różnych danych, mierzony jest czas oraz pamięć. Wyniki są sumowane i zapisywane do pliku *results.txt*. Wyniki są oraz wyświetlane w tabeli z wykresami.

Interfejs użytkownika

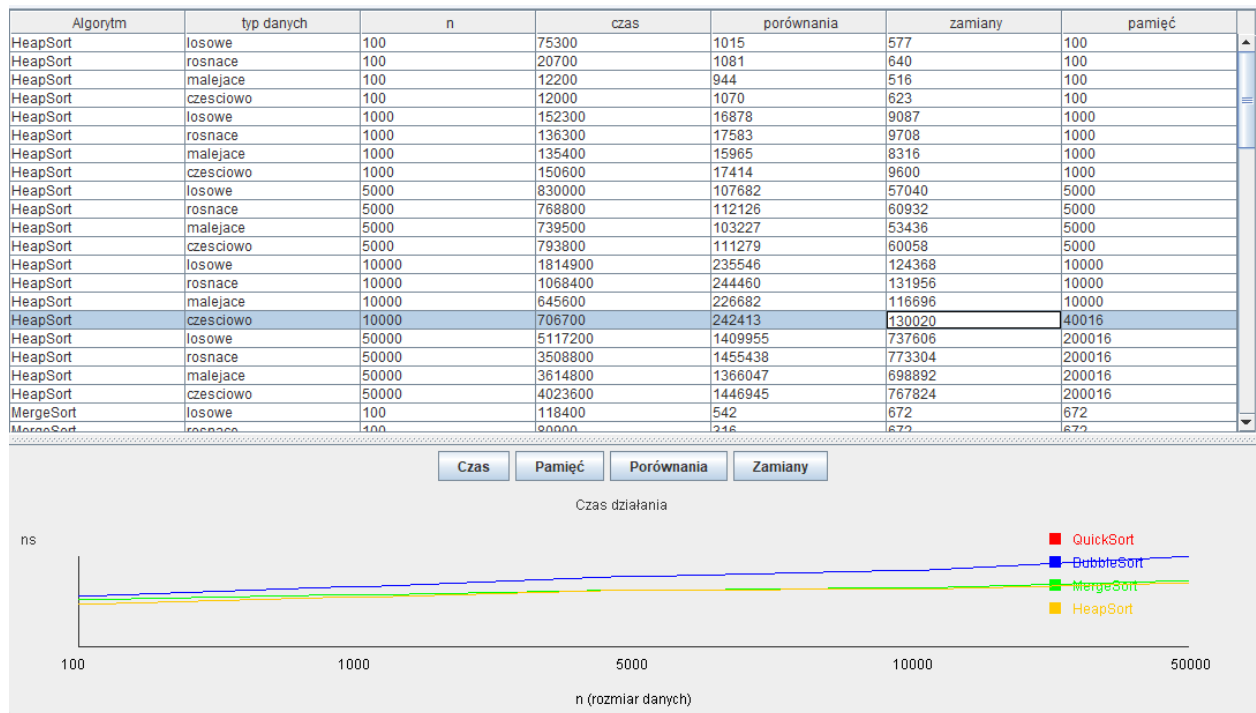
Do rysowanie interfejsu oraz tabel, program wykorzystuje bibliotekę Swing.

Po uruchomieniu programu, jest tworzony przycisk z nadpisem „Uruchom testy”. Po jego naciśnięciu, program rozpoczyna sortowanie różnych typów danych, wykorzystując różne algorytmy.



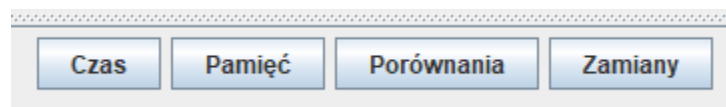
Rysunek 32 - Wyświetlany przycisk do uruchomienia testów

Po zakończeniu sortowania, wyświetlana jest tabela z wynikami oraz różne wykresy. Użytkownik może zobaczyć różne algorytmy wraz z metrykami ich działania na różnych typach danych.



Rysunek 33 - Tabela z wynikami oraz wykresy

Także użytkownika może również wybrać różne wykresy porównujące wszystkie zastosowane algorytmy wykorzystując 4 przyciski.



Rysunek 34 - Przycisk do wybierania wykresów

Generowanie wykresów

Wykresy są rysowane ręcznie przy użyciu bibliotek Java AWT i Swing.

Klasa *Grafiki* dziedziczy po *JPanel*, wykorzystany on jest do rysowania kontenerów, oraz nadpisuje metodę *paintComponent*. Ta metoda jest wykorzystywana do rysowania

oś X i Y, skalowanie wartości, rysowanie linii do każdego algorytmu, dodanie legendy, podpisy osi i tytuł wykresu.

Dane do wykresów przechowywane są w mapach *timeData*, *memoryData*, *swapData* oraz *compData*.

Metoda *updateChart*, która znajduje się w klasie *Main*, usuwa stary wykres, tworzy nowy obiekt klasy *Grafiki* i odświeża wykresy. Użytkownik może przełączać wykresy przyciskami *Czas*, *Pamięć*, *Porównania*, *Zamiany*.

WNIOSKI PROJEKTU

Na podstawie projektu, można zobaczyć, że wydajność algorytmów sortowania w praktyce różni się w zależności od rozmiaru i typu danych. Najlepsze wyniki dostał QuickSort, natomiast MergeSort i HeapSort zapewniły stabilną wydajność. BubbleSort okazał się nieefektywny dla dużych danych.

Uzyskane wyniki są zgodne z oczekiwaniami i pokazują, że dobór odpowiedniego algorytmu ma kluczowe znaczenie dla wydajności programu.

ŹRÓDŁA

Big-O Cheat Sheet, „Złożoność Duże-O typowych algorytmów”, dostęp:
<https://www.bigocheatsheet.com/> [dostęp: 30.03.2026, 16:56]

GeeksforGeeks, „Bubble Sort”, dostęp: <https://www.geeksforgeeks.org/dsa/bubble-sort-algorithm/> [dostęp: 31.03.2026, 11:35]

GeeksforGeeks, „Quick Sort”, dostęp: <https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/> [dostęp: 31.03.2026, 12:51]

GeeksforGeeks, „Merge Sort”, dostęp: <https://www.geeksforgeeks.org/dsa/merge-sort/> [dostęp: 31.03.2026, 19:22]

GeeksforGeeks, „Heap Sort”, dostęp: <https://www.geeksforgeeks.org/dsa/heap-sort/> [dostęp: 07.04.2026, 12:39]

GeeksforGeeks, „Introduction to Java Swing”, dostęp:
<https://www.geeksforgeeks.org/java/introduction-to-java-swing/> [dostęp: 13.04.2026, 20:55]

tutorialspoint, „Java AWT”, dostęp:
https://www.tutorialspoint.com/awt/awt_quick_guide.htm [dostęp: 13.04.2026, 21:40]